

Daily Problem: 10–Feb–2026

Problem Statement

1. Construct a weighted automaton over $\mathbb{R}$ with single-letter alphabet $\{a\}$ that maps a^n to n (i.e. whose weight/series satisfies $f(a^n) = n$ for all $n \geq 0$).
2. Show that no *deterministic* weighted automaton over $\mathbb{R}$ can express this function, where deterministic means: every state has at most one outgoing transition labeled a (equivalently, every input word has a unique run).

Weighted Automata Model

A (real) weighted automaton is a tuple

$$A = (Q, \{a\}, \mu, \lambda, \rho),$$

where Q is a finite set of states, $\mu(a) \in \mathbb{R}^{|Q| \times |Q|}$ is the transition matrix for symbol a , $\lambda \in \mathbb{R}^{1 \times |Q|}$ is the initial row vector, and $\rho \in \mathbb{R}^{|Q| \times 1}$ is the final column vector. It assigns to a^n the weight

$$f(a^n) = \lambda \mu(a)^n \rho \quad (n \geq 0),$$

with $\mu(a)^0 = I$.

Part 1: A weighted automaton mapping $a^n \mapsto n$

Construction

Use two states, so $|Q| = 2$. Define

$$\lambda = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad \rho = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \mu(a) = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}.$$

Verification

We claim that for every $n \geq 0$,

$$\mu(a)^n = \begin{bmatrix} 1 & n \\ 0 & 1 \end{bmatrix}.$$

This follows by induction on n . The base case $n = 0$ gives $\mu(a)^0 = I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$. Assume the formula holds for n . Then

$$\mu(a)^{n+1} = \mu(a)^n \mu(a) = \begin{bmatrix} 1 & n \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & n+1 \\ 0 & 1 \end{bmatrix},$$

which completes the induction.

Therefore,

$$f(a^n) = \lambda \mu(a)^n \rho = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & n \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & n \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = n.$$

So this weighted automaton maps a^n to n , as required.

Part 2: No deterministic weighted automaton can express $a^n \mapsto n$

Deterministic weighted automata on a unary alphabet

Let A be a deterministic weighted automaton over $\{a\}$ in the following (equivalent) sense: each state has *at most one* outgoing transition labeled a . Since the alphabet is unary, this means every input a^n has at most one run, and if the run exists it is unique.

We can model such an automaton as:

- a partial function $\tau : Q \rightarrow Q$ giving the unique a -successor (when it exists),
- a transition weight function $w : Q \rightarrow \mathbb{R}$ giving the weight of the a -edge out of q (when it exists),
- initial weights $\lambda(q)$ and final weights $\rho(q)$.

Because there is a unique run, the weight of a^n is a *single product* along that run:

$$f(a^n) = \sum_{q \in Q} \lambda(q) \cdot \left(\prod_{i=0}^{n-1} w(q_i) \right) \cdot \rho(q_n),$$

where $q_0 = q$ and $q_{i+1} = \tau(q_i)$ (assuming the run is defined). In particular, if there is exactly one initial state with nonzero weight (the common deterministic case), then there is exactly one run from the start state, hence:

$$f(a^n) = \lambda(q_0) \cdot \left(\prod_{i=0}^{n-1} w(q_i) \right) \cdot \rho(q_n).$$

The key point is that the underlying directed graph defined by τ is a *functional graph*: starting from the start state, the visited states form a finite path (a tail) that eventually enters a directed cycle.

Ultimately geometric form of deterministic unary weighted automata

Let the unique run from the start state be

$$q_0 \xrightarrow{a} q_1 \xrightarrow{a} q_2 \xrightarrow{a} \cdots,$$

where $q_{i+1} = \tau(q_i)$. Since Q is finite, there exist indices $0 \leq i < j$ with $q_i = q_j$. Let

$$t = j - i \geq 1$$

be the period of the cycle entered at q_i . Then for all $n \geq i$,

$$q_{n+t} = q_n.$$

Define the cycle-weight product

$$B = \prod_{\ell=0}^{t-1} w(q_{i+\ell}),$$

i.e. the product of transition weights around one full cycle.

Also define the tail-product

$$T = \prod_{\ell=0}^{i-1} w(q_{\ell}),$$

(with the convention $T = 1$ if $i = 0$).

For each residue $r \in \{0, 1, \dots, t-1\}$, define a constant

$$A_r = \lambda(q_0) \cdot T \cdot \left(\prod_{\ell=0}^{r-1} w(q_{i+\ell}) \right) \cdot \rho(q_{i+r}),$$

(where the middle product is 1 if $r = 0$).

Now any $n \geq i$ can be written uniquely as $n = i + kt + r$ with $k \geq 0$ and $0 \leq r < t$. Unrolling the product along the run shows:

$$f(a^{i+kt+r}) = A_r \cdot B^k.$$

So on each residue class modulo t , the function is *geometric* in k .

Why $f(a^n) = n$ is impossible

Assume for contradiction that a deterministic weighted automaton A satisfies

$$f(a^n) = n \quad \text{for all } n \geq 0.$$

Fix the tail/cycle parameters i, t, B above. Consider $n = i + kt + r$ for a fixed residue r . Then the derived form gives

$$i + kt + r = f(a^{i+kt+r}) = A_r \cdot B^k \quad \text{for all } k \geq 0.$$

The left-hand side is an affine (linear) function of k , while the right-hand side is geometric in k . We now distinguish cases:

Case 1: $B = 0$. Then $A_r \cdot B^k = 0$ for all $k \geq 1$, so $f(a^n)$ is eventually 0 along that residue class, contradicting $f(a^n) = n$.

Case 2: $|B| \neq 1$ and $B \neq 0$. Then $|A_r B^k|$ grows (or decays) exponentially in k , while $i + kt + r$ grows linearly in k , impossible for all k .

Case 3: $B = 1$. Then $A_r \cdot B^k = A_r$ is constant in k , so $f(a^{i+kt+r})$ is eventually constant on that residue class. But $i + kt + r$ is unbounded as $k \rightarrow \infty$, contradiction.

Case 4: $B = -1$. Then $A_r \cdot B^k$ alternates between A_r and $-A_r$, hence is bounded. Again this cannot equal the unbounded linear function $i + kt + r$.

All cases lead to a contradiction. Therefore, no deterministic weighted automaton over $\mathbb{R}$ can realize the mapping $a^n \mapsto n$.

Conclusion

- A 2-state (non-deterministic-in-the-matrix sense) weighted automaton with $\mu(a) = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$, $\lambda = [1 \ 0]$, $\rho = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ computes $f(a^n) = n$.
- In contrast, any deterministic unary weighted automaton has ultimately “geometric” behavior on residue classes, so it cannot represent the unbounded linear function n .